

Better Data Transfer

Some of you might be familiar with the famous Czech polyhistor Jára Cimrman. This problem is about one of his lesser-known exploits.

After losing several patents to that pesky Edison who invented the same thing and filed the patent application sooner, Jára decided that he needs to develop the fastest possible way of communicating with the patent office. In order to do this, he came up with two new inventions: one in hardware and one in software.

Hardware invention

As you know, a wire can be used to send a sequence of zeros and ones by switching between low and high voltage. Jára came up with a brilliant idea how to improve this system to speed up data transfer: also using medium voltage to transmit the value “one half”.

The initial experiments with the corresponding hardware changes revealed one potential issue: it’s possible that the new medium voltage pulses cannot be recognized precisely enough if they are as short as the zeros and ones. To be prepared for this alternative, Jára wants to consider a fallback option: he will still use the half-valued bits but they will be transmitted twice as slowly — to give the hardware more time to recognize them.

In the problem, we will use the symbols $\emptyset$, $*$, and 1 to represent the bit values “zero”, “half”, and “one”.

Software invention

Decades before people like Shannon and Huffman, Jára Cimrman already realized that using fixed-width encodings like ASCII is dumb when you care about optimality. He also had a pretty low opinion of the already invented Morse code because it needed spaces between characters, which is also a waste of time. Instead, he developed his own method that would be more efficient.

Task

Jára has a patent application he wants to submit. This application is written in a Moravian dialect, using an alphabet with s different symbols. (For example, if he just used lowercase English letters and spaces, we would have $s = 27$.) As we do not care what the individual symbols are, we will simply number them from 0 to $s - 1$.

You are given the following information and input data:

- You are given the size s of Jára’s alphabet.
- For each symbol i of the alphabet ($0 \leq i < s$) you are given the number a_i of times it appears in Jára’s new patent application.
- You know that when sending a message to the patent office, each 1 or $\emptyset$ (i.e., a bit with a value of zero or one) can be transmitted in one nanosecond.
- You are given the value $h \in \{1, 2\}$ specifying the number of nanoseconds needed to transmit a $*$ (i.e., a bit with the value one half).

A *prefix-free code* for Jára’s alphabet is a mapping such that:

- each symbol i ($0 \leq i < s$) is mapped to a non-empty string x_i
- each x_i consists only of characters 0, *, and 1
- for each $i \neq j$, the string x_i is not a prefix of the string x_j

Note that a prefix-free code allows you to transmit any sequence of original symbols without the need for any separators: if you concatenate the codes of symbols, it is always possible to decode the original symbols without any ambiguity.

Construct a prefix-free code that would have been the best one to encode Jára's patent application.

In each subproblem a **partial score** of 50% of its points is awarded for just computing the smallest time (in nanoseconds) needed to transmit the encoded message, without constructing a corresponding code.

Example of the encoding

Suppose that Jára uses $s = 7$ distinct symbols. One possible prefix-free code for these seven symbols is to map each 0 to the string "011", each 1 to the string "0*", each 2 to the string "00", each 3 to the string "010", each 4 to the string "1", each 5 to the string "01*", and finally each 6 to the string "*".

A message we want to send is a sequence of symbols from the given alphabet, for example, (0, 0, 6, 2, 0). We can encode this message into bits by simply replacing each symbol with its code and concatenating those codes: "011" + "011" + "*" + "00" + "011" = "011011*00011".

The recipient of the message "011011*00011" will now be able to split this encoded message back into the individual strings in a unique way, and from those they can reconstruct the original message.

If $h = 2$, we would need 13 nanoseconds to transmit this encoded message.

Input format

The first line of input contains the integer $t \leq 500$: the number of test cases that follow.

Each test case consists of two lines. The first line contains the values s and h . The second line contains the integers $a_0, \dots, a_{s-1}$.

In each test case, for each i , $1 \leq a_i \leq 10^9$.

Output format

For each test case, output two lines. The first line should contain the optimal time (in nanoseconds) in which it was possible to transmit Jára's new patent application by choosing the best possible prefix-free code. The second line should contain one optimal prefix-free code that corresponds to that optimal time. That is, the second line should contain the s whitespace-separated non-empty strings $x_0, \dots, x_{s-1}$.

(You must always print two lines, even if only attempting to get the partial score. The content of the second line is ignored for that purpose.)

Subproblem B1 (10 points, public)

Input file: [B1.in \(/static/online2026/B1.d3924e18c379413122b5aa2fe9878953.in\)](/static/online2026/B1.d3924e18c379413122b5aa2fe9878953.in).

Constraints: $t = 100$ and in each test case $1 \leq s \leq 10$ and $h = 1$.

Subproblem B2 (16 points, public)

Input file: [B2.in \(/static/online2026/B2.6dd6d42199257248e77ff52e017d759d.in\)](/static/online2026/B2.6dd6d42199257248e77ff52e017d759d.in)

Constraints: $t = 100$ and in each test case $1 \leq s \leq 10$ and $h = 2$.

Subproblem B3 (30 points, secret)

Input file: [B3.in \(/static/online2026/B3.fbec6bc035e6ef0e59ad12e012d5fe8b.in\)](/static/online2026/B3.fbec6bc035e6ef0e59ad12e012d5fe8b.in)

Constraints: $t = 100$ and in each test case $1 \leq s \leq 2\,000$ and $h = 1$.

Subproblem B4 (24 points, secret)

Input file: [B4.in \(/static/online2026/B4.5fc1a0b88964524b2ac30bfc79d1baf1.in\)](/static/online2026/B4.5fc1a0b88964524b2ac30bfc79d1baf1.in)

Constraints: $t = 500$ and in each test case $1 \leq s \leq 70$.

Subproblem B5 (20 points, secret)

Input file: [B5.in \(/static/online2026/B5.026668e5be99bb1cc15bb6554f178126.in\)](/static/online2026/B5.026668e5be99bb1cc15bb6554f178126.in)

Constraints: $t = 500$ and in each test case $1 \leq s \leq 200$.

Examples

input	output
<pre> 6 3 1 100 80 90 3 2 100 80 90 1 2 100 4 2 100 80 10 90 7 1 3 141 59 2 653 58 979 7 2 3 141 59 2 653 58 979 </pre>	<pre> 270 0 1 * 350 1 * 0 100 1 460 0 * 10 11 2221 011 0* 00 010 1 01* * 2545 *11 *0 ** *1* 1 *10 0 </pre>

Example descriptions:

Test 1: Each of the three symbols of the alphabet can be mapped to a different 1-character string, the order does not matter.

Test 2: Here we have to map the least frequent symbol to the * character that takes longer to transmit. The total transmission time will be $100 * 1 + 80 * 2 + 90 * 1$ nanoseconds.

Test 3: By mapping the only symbol to 1 (or equivalently 0) we can transfer the whole message in 100 nanoseconds.

Test 4: We will need 1 nanosecond to transmit symbol 0 (which we'll do 100 times). Each of the other three symbols of the alphabet is encoded into a string that takes 2 nanoseconds to transmit.